Async/Await & Error Handling in NestJS

Nov 4, 2026 · @Nam

Overview

NestJS uses async/await just as much as plain Node, because it runs on Express (or Fastify) underneath. What changes is the structure around it: modules, dependency injection and decorators.

Nest also takes over the plumbing you write by hand in Express: sending the response, setting status codes and catching errors. As a result, you write far fewer `try/catch` blocks, and only where you actually do something with the error.

A standard controller and service

In Nest, the service holds the business logic and data access, and the controller handles only HTTP. The examples use TypeORM; the pattern is the same with Prisma.

Express, for comparison

```
router.get('/appointments/:id', async (req, res, next) => {
  try {
    const appt = await db.appointment.findById(req.params.id);
    if (!appt) return res.status(404).json({ message: 'Not found' });
    res.json(appt);
  } catch (err) {
    next(err);
  }
});
```

Service

```
// appointments.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Appointment } from './appointment.entity';
import { CreateAppointmentDto } from './dto/create-appointment.dto';
```

```
@Injectable()
export class AppointmentsService {
  constructor(
    @InjectRepository(Appointment)
    private readonly repo: Repository<Appointment>,
  ) {}

  findAll(): Promise<Appointment[]> {
    return this.repo.find();
  }

  async findOne(id: number): Promise<Appointment> {
    const appt = await this.repo.findOneBy({ id });
    if (!appt) throw new NotFoundException(`Appointment ${id} not found`);
    return appt;
  }

  async create(dto: CreateAppointmentDto): Promise<Appointment> {
    const appt = this.repo.create(dto);
    return this.repo.save(appt);
  }
}
```

Controller

```
// appointments.controller.ts
import { Body, Controller, Get, Param, ParseIntPipe, Post } from
'@nestjs/common';
import { AppointmentsService } from './appointments.service';
import { CreateAppointmentDto } from './dto/create-appointment.dto';

@Controller('appointments')
export class AppointmentsController {
  constructor(private readonly appointmentsService: AppointmentsService) {}

  @Get()
  findAll() {
    return this.appointmentsService.findAll();
  }

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
```

```
    return this.appointmentsService.findOne(id);
  }

  @Post()
  create(@Body() dto: CreateAppointmentDto) {
    return this.appointmentsService.create(dto);
  }
}
```

Module wiring

```
@Module({
  imports: [TypeOrmModule.forFeature([Appointment])],
  controllers: [AppointmentsController],
  providers: [AppointmentsService],
})
export class AppointmentsModule {}
```

What Nest handles for you

- **It waits for promises.** A handler can return a plain value, a Promise or an RxJS Observable. Nest waits for the result and sends it as JSON (200 by default, 201 for `@Post`). That is why most controller methods above don't say `async`.
- **Thrown exceptions become HTTP responses.** `throw new NotFoundException()` inside an async service becomes a 404 with a JSON body, through Nest's built-in exception filter. No `try/catch` or `next(err)` in every route.
- **You don't touch `req` / `res`.** `@Param`, `@Body`, `@Query` and `@Headers` pull values from the request. Pipes such as `ParseIntPipe`, or a global `ValidationPipe` with class-validator DTOs, convert and validate before your method runs.
- **No manual `new`.** Dependency injection passes the service into the controller and the repository into the service, which makes them easy to mock in tests.

Avoid injecting `@Res()` unless you really need it. It makes you send the response yourself, so you lose auto-serialization and interceptors. Use `@HttpCode()` and `@Header()` instead.

When you do need async/await

- When you use the result before returning it, as `findOne` does with its `null` check.

- Inside `try/catch`: write `return await`. A bare `return promise` lets the rejection escape the catch block.
- When you run several calls in sequence or in parallel (`await Promise.all([...])`).

Where to use try/catch

Use `try/catch` only when you will do something with the error. Nest's exception filter already catches anything thrown from a controller, service or pipe.

1. Translating low-level errors into HTTP errors

This is the most common case. An unhandled database error becomes a generic 500, so catch the failures that mean something to the client and rethrow them as Nest exceptions.

```
async create(dto: CreatePatientDto): Promise<Patient> {
  try {
    return await this.repo.save(this.repo.create(dto));
  } catch (err) {
    if (err instanceof QueryFailedError && (err as any).code === '23505') {
      throw new ConflictException('A patient with this email already
exists');
    }
    throw err; // anything unexpected: let it bubble up as a 500
  }
}
```

- `return await` is required. Without `await`, the promise rejects after the function has returned and the catch never runs.
- Always rethrow what you didn't handle. Swallowing unknown errors hides bugs.
- `23505` is Postgres's unique-violation code; other databases use different codes.

2. Calls to external services

Payment gateways, SMS or email providers and third-party APIs. Their failures shouldn't reach the client as a raw 500 with an Axios stack trace.

```
async sendReminder(appt: Appointment) {
  try {
    await this.smsClient.send(appt.patient.phone, `Reminder: ${appt.time}`);
  } catch (err) {
    this.logger.warn(`SMS failed for appointment ${appt.id}:
${err.message}`);
    // Swallow it (a reminder isn't critical) or
    // throw new ServiceUnavailableException('SMS provider unavailable');
  }
}
```

This is one of the few places where swallowing an error can be correct: when the side effect isn't critical to the request.

3. Cleanup or rollback

When something must happen whether the call succeeds or fails.

```
const queryRunner = this.dataSource.createQueryRunner();
await queryRunner.connect();
await queryRunner.startTransaction();
try {
  await queryRunner.manager.save(appointment);
  await queryRunner.manager.save(invoice);
  await queryRunner.commitTransaction();
} catch (err) {
  await queryRunner.rollbackTransaction();
  throw err;
} finally {
  await queryRunner.release();
}
```

TypeORM's `dataSource.transaction(async (manager) => { ... })` and Prisma's `$transaction` handle the rollback for you, so prefer those when you can.

4. Code outside the request cycle

Scheduled jobs (`@Cron`), queue consumers, event listeners and any promise you don't `await` have no HTTP exception filter behind them. An uncaught error there is lost or crashes the process, so wrap the job body.

```
@Cron('0 8 * * *')
async sendDailyReminders() {
  try {
    // ...
  } catch (err) {
    this.logger.error('Daily reminder job failed', err.stack);
  }
}
```

Where not to use try/catch

- **In controllers.** A `try/catch` there almost always belongs in the service or an exception filter.
- **Around your own `throw new NotFoundException()`.** Let it propagate.
- **Catch, log and rethrow the same error.** Nest already logs unhandled errors. For logging or error formatting everywhere, write one global exception filter.

Global exception filter for repeated translations

If several services catch `23505` and map it to a 409, move that into a filter and register it globally.

```
@Catch(QueryFailedError)
export class DbExceptionFilter implements ExceptionFilter {
  catch(err: QueryFailedError, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    if ((err as any).code === '23505') {
      return res.status(409).json({ statusCode: 409, message: 'Duplicate
entry' });
    }
    res.status(500).json({ statusCode: 500, message: 'Database error' });
  }
}

// main.ts
app.useGlobalFilters(new DbExceptionFilter());
```

Then the services need no `try/catch` for that case. Keep a local one only when you need a specific message, like "a patient with this email already exists".

Exception strategy: three approaches

Preferred: B2, domain errors in services, mapped to HTTP by one global exception filter. The services stay free of HTTP concepts, each mapping is written once, and new routes are covered automatically.

Approach	Service throws	Mapped to HTTP by	Pros	Cons
A	`NotFoundException` (Nest HTTP exception)	Nest's built-in filter	Least code; common in Nest projects	Services coupled to HTTP; cron jobs, queue workers, GraphQL and other services receive HTTP-shaped errors
B1	`AppointmentNotFoundError` (domain error)	`try/catch` in each controller	Services transport-agnostic; mapping visible per route; each route can map differently	Mapping repeated per route; a route that forgets it returns 500; responses can drift between controllers
B2 (preferred)	`AppointmentNotFoundError` (domain error)	One global exception filter	Services transport-agnostic; each mapping written once; new routes covered automatically	Mapping is less visible at the route; per-route differences need a local override

A: HTTP exceptions in services

```
// service
if (!appt) throw new NotFoundException('Appointment not found');
// controller: just returns the service call
```

B1: domain errors, mapped in the controller

```
// errors/appointment-not-found.error.ts
export class AppointmentNotFoundError extends Error {}

// service
if (!appt) throw new AppointmentNotFoundError(`Appointment ${id} not found`);

// controller
@Get(':id')
async findOne(@Param('id', ParseIntPipe) id: number) {
  try {
    return await this.appointmentsService.findOne(id);
  } catch (err) {
    if (err instanceof AppointmentNotFoundError) {
      throw new NotFoundException(err.message, { cause: err });
    }
    throw err;
  }
}
```

This is the pattern in the second example of the [NestJS exception filters docs]. Use it only for a route that must map an error differently from the global filter. When you do:

- Use `return await` inside the `try`, or the catch never runs.
- Always rethrow errors you didn't map.
- Every route that calls the service needs the mapping; a missing one surfaces as a 500, so cover each route's error cases in tests.

B2: domain errors, mapped in a global filter (preferred)

```
@Catch(AppointmentNotFoundError)
export class DomainExceptionFilter implements ExceptionFilter {
  catch(err: AppointmentNotFoundError, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    res.status(404).json({ statusCode: 404, message: err.message });
  }
}
// main.ts: app.useGlobalFilters(new DomainExceptionFilter());
```

Register the filter once in `main.ts` and every route gets the same mapping. For a route that needs a different status code, override it locally with a B1-style `try/catch`.

Why option A limits reuse

`NotFoundException` says both what went wrong and how to report it over HTTP. When a cron job, queue worker or another service calls the same method, it receives a 404 that means nothing in its context, and the service has picked a status code for a caller it knows nothing about.

Whichever approach the codebase uses, use it consistently: mixing HTTP exceptions and domain errors across services makes it unclear which errors are already HTTP-shaped.